

UNIVERSITÀ DEGLI STUDI DI PADOVA

Corso di Laurea in Informatica
Insegnamento di Ingegneria del Software

Anno Accademico 2025/2026

Specifica Tecnica

Architettura e Design di Sistema

Gruppo: Synergo Unit

Email: synergounit20@gmail.com

Versione: **v0.3.1**
Data: **2026-08-06**

Registro delle Modifiche

Versione	Data	Autore	Verificatore	Descrizione
0.3.1	2026-08-06	Nome Cognome	Nome Cognome	Editing di forma su Introduzione (rimosso paragrafo ridondante rispetto allo Scopo del documento; tolto dettaglio non necessario sul Tedesco; riformulata la descrizione del cappello blu) e Tecnologie (corretto lo strumento di modellazione UML in StarUML; aggiunta nota su DI e pattern non forniti nativamente da Vue.js).
0.3.0	2026-08-06	Nome Cognome	Nome Cognome	Rimossa la persistenza server-side (PostgreSQL) dallo stack tecnologico e dall'architettura di deployment, non implementata in questa fase. Requisiti opzionali correlati (R4-V-Op, R7/R8-Q-Op, R78–R86) riportati con stato “Non implementato” e relativa motivazione nella Sezione 7.
0.2.0	2026-08-06	Nome Cognome	Nome Cognome	Disciplina dei riferimenti: URL puntuali (non generici) verso la documentazione di terze parti con data di ultimo accesso; citazioni interne circoscritte alla sezione rilevante di Analisi dei Requisiti e Analisi dello Stato dell'Arte, come richiesto in sede di revisione.

Continua nella pagina successiva

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026-08-06	Nome Cognome	Nome Cognome	Prima stesura completa del documento: tecnologie, architettura logica e di deployment, interfaccia API, design pattern, flusso dati e tracciamento dei requisiti, a partire da Analisi dello Stato dell'Arte v1.0.0 e Analisi dei Requisiti v1.8.0.
0.0.0	YYYY-MM-DD	Nome Cognome	Nome Cognome	Prima stesura della struttura del documento.

Indice

1	Introduzione	7
1.1	Scopo del documento	7
1.2	Scopo del prodotto	7
1.3	Glossario	7
1.4	Riferimenti	8
1.4.1	Normativi	8
1.4.2	Informativi	8
2	Tecnologie Utilizzate	9
2.1	Linguaggi di programmazione	9
2.2	Framework	9
2.3	Librerie	9
2.4	Persistenza dei dati	9
2.5	Strumenti di sviluppo	9
2.6	Motivazioni delle scelte tecnologiche	10
3	Architettura del Sistema	11
3.1	Architettura Generale	11
3.2	Architettura di Deployment	11
3.3	Architettura del Frontend	11
3.3.1	Gestione dello Stato e dell'Editor	12
3.3.2	Gestione Interazioni AI (Widget/Pannello AI)	12
3.4	Architettura del Backend	13
3.4.1	Livello di Presentazione / Controller	13
3.4.2	Livello di Business Logic (Core LLM)	14
3.4.3	Livello di Integrazione / Infrastructure	14
4	Interfaccia API e Comunicazione	16
4.1	Struttura dei Payload AI	16
4.2	Gestione degli Errori e Fallback	16
5	Design Pattern Utilizzati	18
5.1	Pattern Creazionali	18
5.1.1	Factory Method (con auto-registrazione)	18
5.1.2	Builder	18
5.2	Pattern Strutturali	19
5.2.1	Decorator	19

5.2.2	Adapter	19
5.2.3	Proxy (remoto)	20
5.3	Pattern Comportamentali	20
5.3.1	Strategy	20
5.3.2	Command	20
5.3.3	Observer	21
5.3.4	Dependency Injection	21
6	Flusso dei Dati (Data Flow)	22
6.1	Flusso di esecuzione: Elaborazione AI (es. riassunto)	22
6.2	Flusso di esecuzione: Editing con Undo	23
6.3	Flusso di esecuzione: Salvataggio locale della nota	23
7	Tracciamento e Stato dei Requisiti	25
7.1	Tabella Tracciamento Componenti-Requisiti	25
7.2	Stato di Soddisfacimento dei Requisiti	26

Elenco delle figure

1	Architettura di alto livello e di deployment del sistema.	11
2	Diagramma dei package del Frontend.	12
3	Diagramma delle classi del livello Model del Frontend.	12
4	Diagramma delle classi dei livelli View, Controller e Proxy del Frontend.	13
5	Diagramma dei package del Backend.	13
6	Diagramma delle classi del livello di presentazione e DI del Backend.	14
7	Diagramma delle classi: operazioni AI e costruzione dei prompt (<code>AI_Domain/domain</code>).	14
8	Diagramma delle classi: integrazione con il servizio LLM (<code>AI_Domain/llm</code>).	15
9	Diagramma di sequenza: richiesta di un'operazione AI, tratto Frontend.	22
10	Diagramma di sequenza: richiesta di un'operazione AI, tratto REST → LLM.	23
11	Diagramma di sequenza: formattazione di un testo e relativo Undo.	23
12	Diagramma di sequenza: salvataggio di una nota in locale.	24

Elenco delle tabelle

1	Tracciamento Componenti → Requisiti	25
2	Stato dei Requisiti	26

1 Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di definire l'architettura di base e di dettaglio del sistema **Second Brain**. Vengono illustrate le scelte tecnologiche effettuate dal gruppo Synergo Unit durante la fase di Product Baseline, i pattern architetturali e di design adottati, la decomposizione in componenti del sistema e il tracciamento bidirezionale tra queste componenti e i requisiti individuati nella fase di Analisi.

A differenza della Technology Baseline, in cui l'obiettivo era esclusivamente validare la fattibilità tecnica delle scelte effettuate tramite un Proof of Concept, questo documento consolida un'architettura definitiva, coerente con il PoC ma estesa e rifinita per coprire la totalità dei requisiti obbligatori individuati in [Analisi dei Requisiti](#) (Sezione 4 - Requisiti).

1.2 Scopo del prodotto

Second Brain è un'applicazione web per la redazione di testi in formato Markdown, pensata per superare i limiti dei classici chatbot testuali fornendo un editor integrato in cui un Large Language Model (LLM) può attivamente elaborare, criticare e generare parti di testo su richiesta dell'utente.

Le funzionalità principali riguardano:

- la scrittura e formattazione di testo Markdown (grassetto, corsivo, sottolineato, barrato, citazioni, intestazioni, link, elenchi, tabelle) con anteprima in tempo reale e cronologia di annullamento/ripristino;
- il supporto AI alla scrittura: riassunto, traduzione (Inglese, Francese, Spagnolo) e riscrittura del testo selezionato;
- l'analisi critica del testo secondo il metodo dei "Sei Cappelli per Pensare" di Edward de Bono (dati e fatti, emozioni, rischi, opportunità, creatività, logica/controllo);
- il *Distant Writing*: la generazione di interi blocchi di testo a partire da un prompt fornito dall'utente in una finestra dedicata;
- la gestione dei dati: salvataggio, apertura ed esportazione (PDF, HTML, JSON) delle note come requisito obbligatorio basato sul file system locale, con gestione di note remote, autenticazione utente e persistenza server-side come estensione opzionale.

L'utente di riferimento è un individuo senza competenze tecniche pregresse (tipicamente uno studente) che utilizza l'applicazione per prendere appunti e desidera supporto nella loro elaborazione tramite LLM; l'interfaccia deve pertanto risultare semplice e intuitiva. L'unico attore secondario del sistema è il servizio LLM esterno, richiamato tramite API compatibile con lo standard OpenAI.

1.3 Glossario

Al fine di evitare ambiguità, i termini tecnici o con significato specifico sono seguiti dal pedice e definiti nel documento [Glossario](#).

1.4 Riferimenti

1.4.1 Normativi

- [Capitolato C6 – Second Brain \(Zucchetti S.p.A.\)](#)
Ultimo accesso: 2026-08-06;
- [Norme di Progetto, v2.12.0;](#)

1.4.2 Informativi

Documenti interni del progetto, citati per versione (ogni versione è una revisione immutabile, già registrata nel relativo registro delle modifiche):

- [Analisi dei Requisiti](#), v1.8.0, in particolare la Sezione 3 (Casi d'Uso) e la Sezione 4 (Requisiti);
- [Analisi dello Stato dell'Arte](#), v1.0.0, in particolare la Sezione 2 (Tecnologie Frontend), la Sezione 3 (Tecnologie Backend) e la Sezione 4 (Struttura Tecnica Preliminare del Proof of Concept);
- [Piano di Qualifica, v1.7.0;](#)

Documentazione tecnica di terze parti, citata puntualmente alla pagina rilevante; trattandosi di risorse web soggette a variazione, ciascuna riporta la data di ultimo accesso:

- [Vue.js Guide – Reactivity Fundamentals](#)
Ultimo accesso: 2026-08-06;
- [CodeMirror – System Guide](#)
Ultimo accesso: 2026-08-06;
- [FastAPI – Dependencies](#)
Ultimo accesso: 2026-08-08;
- [OpenAI API Reference](#), pagina *Create chat completion* – standard di riferimento per l'interazione con i modelli LLM (R2-V-O)
Ultimo accesso: 2026-08-08;
- [Docker Docs – Docker Compose](#)
Ultimo accesso: 2026-08-03;

2 Tecnologie Utilizzate

Le tecnologie qui descritte confermano e consolidano le scelte motivate in dettaglio in [Analisi dello Stato dell'Arte](#) Sezioni 2–3, a cui si rimanda per l'analisi comparativa completa delle alternative scartate.

2.1 Linguaggi di programmazione

Il Frontend è sviluppato in **TypeScript**, per la tipizzazione statica che riduce gli errori di integrazione con le interfacce dei payload AI. Il Backend è sviluppato in **Python**, scelto per la maturità del suo ecosistema nell'integrazione con strumenti di intelligenza artificiale e NLP e per la conoscenza pregressa del linguaggio da parte di alcuni membri del gruppo.

2.2 Framework

- **Vue.js** (Frontend): framework a componenti scelto per la progressività (si parte da codice semplice, aggiungendo complessità solo dove necessario), la separazione netta tra struttura HTML e styling CSS e la bassa curva di apprendimento, coerente con l'esigenza di un'interfaccia semplice e intuitiva anche per utenti non tecnici.
- **FastAPI** (Backend): framework per API REST asincrono, scelto per le prestazioni elevate e il supporto nativo alle chiamate asincrone, fondamentale per non bloccare il servizio durante l'attesa delle risposte dell'LLM. Fornisce inoltre validazione dei dati integrata, generazione automatica della documentazione (Swagger/OpenAPI) e gestione di middleware per logging, sicurezza e rate limiting.

2.3 Librerie

- **CodeMirror**: componente editor di testo integrato nel Frontend, scelto per il bilanciamento tra leggerezza ed estensibilità. Fornisce un ecosistema di estensioni pronte per Markdown, syntax highlighting e keybinding, oltre alla possibilità di integrare comandi personalizzati per l'interazione diretta con l'LLM dalla finestra di scrittura.
- Librerie di validazione dati e generazione di schemi (integrate in FastAPI tramite Pydantic) per la definizione tipizzata dei payload delle operazioni AI (Sezione 4).

2.4 Persistenza dei dati

Il gruppo ha scelto di non implementare alcun sistema di persistenza server-side in questa fase: la gestione dei dati si basa esclusivamente sul file system locale del browser (salvataggio, apertura ed esportazione della nota, R75–R77), che copre i requisiti obbligatori del capitolato. La persistenza remota delle note e la gestione di utenti autenticati sono funzionalità opzionali (R4-V-Op, R78–R86) che il gruppo ha deciso di non realizzare, per concentrare le risorse disponibili sul consolidamento dei requisiti obbligatori; non è pertanto presente alcun database nello stack tecnologico né nell'architettura di deployment (Sezione 3.2). Lo stato di questi requisiti opzionali è riportato in dettaglio in Sezione 7.

2.5 Strumenti di sviluppo

- **Docker Compose** per la containerizzazione e l'esecuzione coordinata di Frontend e Backend, scelto rispetto a un'installazione manuale (poco riproducibile) o a una macchina

virtuale (overhead eccessivo per le esigenze del progetto); riduce le differenze tra ambienti locali e semplifica l'avvio del sistema con un unico comando.

- **Git** come strumento di versionamento del codice, con tracciamento delle attività tramite issue (requisito R5-Q-O).
- **StarUML** per la modellazione UML e la produzione dei diagrammi architetturali.

2.6 Motivazioni delle scelte tecnologiche

Le alternative scartate e i relativi motivi sono riportati integralmente in [Analisi dello Stato dell'Arte](#), Sezioni 2–3; si riepilogano qui i punti salienti:

- **Angular** e **React** sono stati scartati a favore di Vue.js rispettivamente per l'eccessivo peso/complessità rispetto agli obiettivi del sistema e per il rischio di debito tecnico iniziale dato dall'assenza di standard architetturali consolidati nel team. La contropartita di questa scelta è che Vue.js, a differenza di Angular, non fornisce nativamente un sistema di Dependency Injection né impone pattern architetturali: la struttura MVC/Observer descritta in Sezione 3.3 è stata quindi progettata e implementata interamente dal gruppo, senza supporto diretto dal framework;
- **Monaco Editor** è stato scartato a favore di CodeMirror per l'elevato consumo di risorse e la configurazione più complessa;
- **Django** e **Flask** sono stati scartati a favore di FastAPI: Django per la natura sincrona e il peso eccessivo rispetto a un sistema che deve restare semplice, Flask per la mancanza nativa di validazione dei dati e supporto asincrono.

Un vincolo trasversale a tutte le scelte tecnologiche è l'interazione con i modelli LLM (Gemma, ChatGPT-OSS, Qwen) tramite API aderenti allo [standard OpenAI](#) (R2-V-O, R3-V-O): questo vincolo, unito alla necessità di poter sostituire il provider LLM senza impatti sul resto del sistema, ha guidato l'adozione del pattern Adapter descritto in Sezione 5.

3 Architettura del Sistema

3.1 Architettura Generale

Il sistema *Second Brain* è basato su un'architettura di tipo Client-Server per garantire la separazione delle responsabilità (Separation of Concerns) tra la gestione dell'interfaccia utente (editor Markdown) e l'orchestrazione delle chiamate ai Modelli di Linguaggio di Grande Scala (LLM), in linea con il vincolo R1-V-O.

Il Frontend Vue.js gestisce l'intera interazione con l'utente e non comunica mai direttamente con il servizio LLM: ogni richiesta di elaborazione AI transita dal Backend FastAPI, che funge da unico punto di orchestrazione verso il provider esterno. Questa scelta, oltre a rispettare il vincolo R2-V-O sullo standard delle chiamate API, evita di esporre nel browser eventuali credenziali di accesso al servizio LLM.

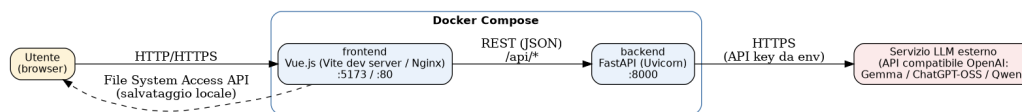


Figura 1: Architettura di alto livello e di deployment del sistema.

3.2 Architettura di Deployment

Frontend e Backend sono eseguiti in due container Docker distinti, orchestrati tramite Docker Compose (Figura 1):

- **frontend:** applicazione Vue.js, servita in sviluppo dal dev server Vite e in produzione da un web server statico;
- **backend:** servizio FastAPI eseguito tramite Uvicorn, espone le API REST consumate dal frontend e contatta il servizio LLM esterno tramite HTTPS, con la relativa API key iniettata come variabile d'ambiente.

Non è presente alcun container di persistenza: come descritto in Sezione 2.4, il gruppo ha deciso di non implementare la persistenza remota, funzionalità opzionale del capitolato (R4-V-Op, R78–R86); tutti i dati sono gestiti tramite il file system locale del browser, senza alcun componente server-side stateful.

Coerentemente con quanto indicato in [Analisi dello Stato dell'Arte](#), Sezione 4, la separazione in container non costituisce di per sé una scelta architetturale definitiva quanto piuttosto uno strumento per rendere l'ambiente di esecuzione riproducibile, portabile e semplice da avviare con un unico comando.

3.3 Architettura del Frontend

Il Frontend gestisce l'interazione dell'utente con l'editor Markdown e la visualizzazione dello stato delle operazioni AI. Il pattern architetturale adottato è **Model-View-Controller** nella forma basata su Observer: il Model realizza un'interfaccia **Subject** (Observable); la View realizza sia **Subject** sia **Observer**, potendo quindi essere osservata a sua volta dal Controller; il Controller realizza **Observer**. Il Controller osserva la View, la View osserva il Model; il Controller mantiene inoltre un riferimento diretto al Model per invocare comandi, senza però riceverne le notifiche direttamente.

Questa separazione esplicita, indipendente dal framework di rendering, mantiene la logica di dominio (Model e Controller) testabile in isolamento: i componenti Vue.js costituiscono l'implementazione concreta della View, limitandosi a riflettere nel template lo stato esposto dal relativo oggetto View e a inoltrare gli eventi dell'utente, senza contenere essi stessi logica applicativa.

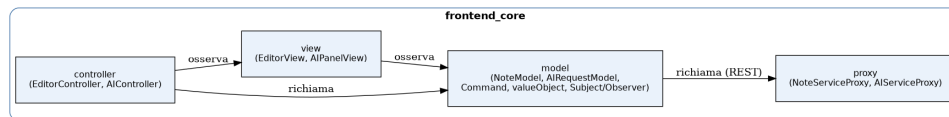


Figura 2: Diagramma dei package del Frontend.

3.3.1 Gestione dello Stato e dell'Editor

Il `NoteModel` mantiene lo stato osservabile della nota corrente (contenuto, identificativo, flag di modifica non salvata) e delega la manipolazione testuale a un `MarkdownContentEditor`. Ogni modifica (formattazione, tabelle, elenchi, link, inserimento di testo) è incapsulata in un comando (Sezione 5) eseguito tramite `NoteModel.executeCommand()`, che aggiorna una `CommandHistory` a due pile per supportare Undo/Redo (R43-F-O, R44-F-O) in modo uniforme per tutte le operazioni di editing. Il salvataggio e l'apertura della nota (R75-F-O, R76-F-O) sono delegati tramite l'interfaccia `NoteService` a un'implementazione basata sul File System locale, così da non vincolare il Model a un meccanismo di persistenza specifico.

3.3.2 Gestione Interazioni AI (Widget/Pannello AI)

L'`AIRequestModel` mantiene lo stato della richiesta AI corrente come union type (`Idle`, `Processing`, `ProposalReady`, `Error`), riflesso dal componente `AIPanelView` tramite il meccanismo Observer (R72-F-O: visualizzazione continua dello stato di avanzamento). L'utente può accettare, rifiutare, rigenerare o interrompere manualmente una proposta AI (R69–R71, R73-F-O): l'accettazione riusa lo stesso meccanismo di comando dell'editing manuale, inserendo il testo proposto tramite un comando eseguito su `NoteModel` (Sezione 5, pattern Command). Le richieste verso il Backend sono mediate dall'interfaccia `AIService`, la cui implementazione concreta (Proxy) incapsula i dettagli della chiamata REST.

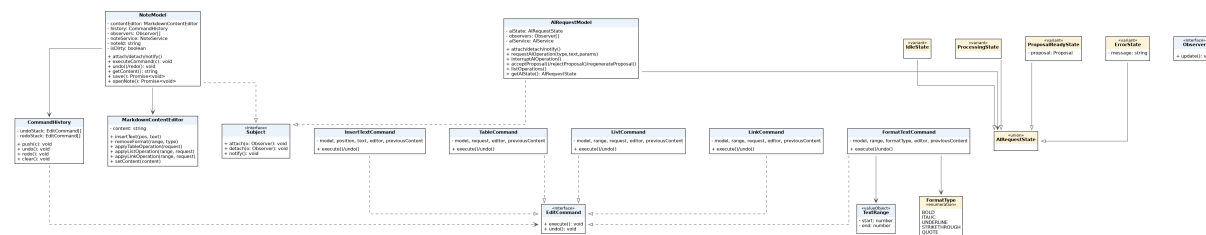


Figura 3: Diagramma delle classi del livello Model del Frontend.

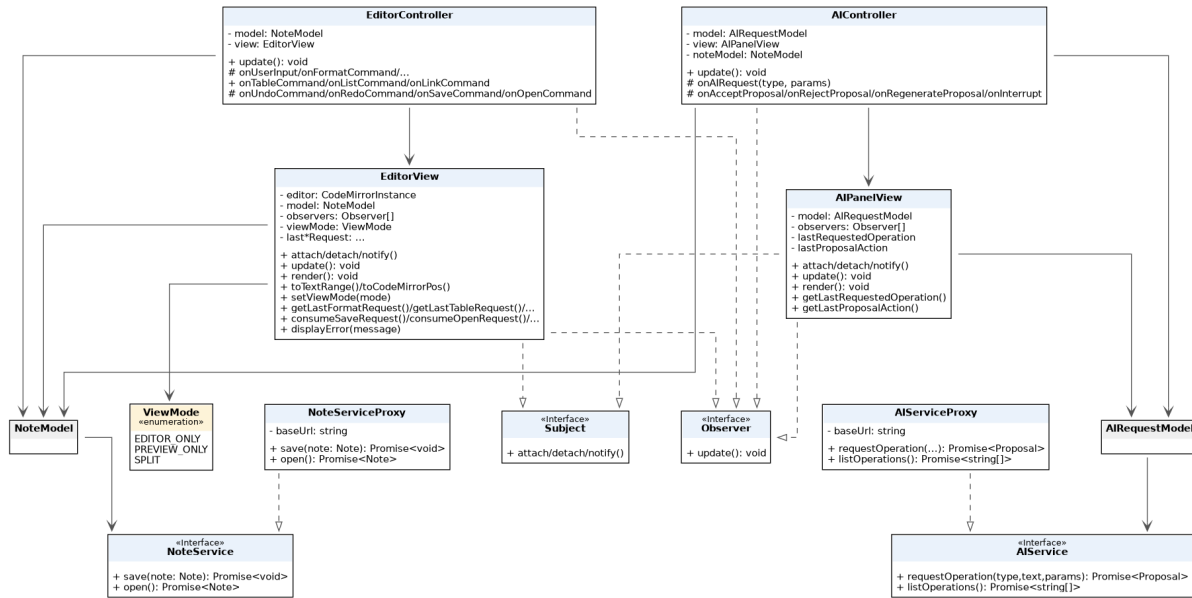


Figura 4: Diagramma delle classi dei livelli View, Controller e Proxy del Frontend.

3.4 Architettura del Backend

Il Backend funge da orchestratore centrale e da strato di astrazione verso i provider LLM esterni. È organizzato in due package Python distinti: il livello applicativo/di presentazione (router REST, DTO, Dependency Injection) e il dominio AI (**AI_Domain**), che contiene la logica di costruzione dei prompt e di interazione con l'LLM indipendente dal framework web.

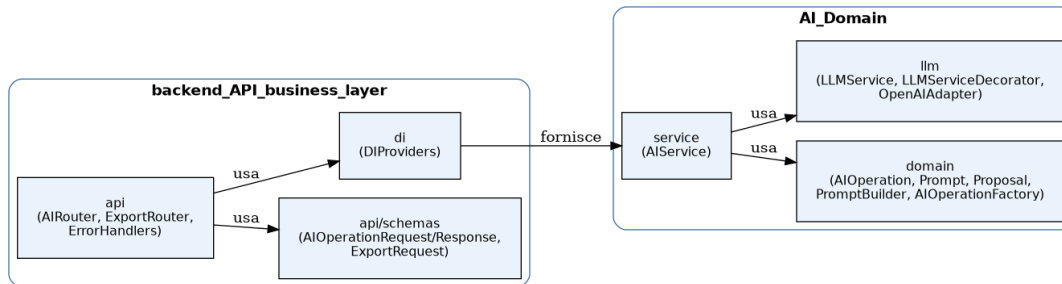


Figura 5: Diagramma dei package del Backend.

3.4.1 Livello di Presentazione / Controller

I router FastAPI (**AIRouter**, **ExportRouter**) ricevono le richieste dal client, le validano tramite i DTO di **api/schemas** (**AIOperationRequest**, **ExportRequest**) e delegano l'elaborazione ai servizi applicativi, ottenuti tramite Dependency Injection (**DIProviders**). Un gestore di errori centralizzato (**ErrorHandlers**) traduce le eccezioni di dominio in risposte HTTP coerenti (Sezione 4.2).

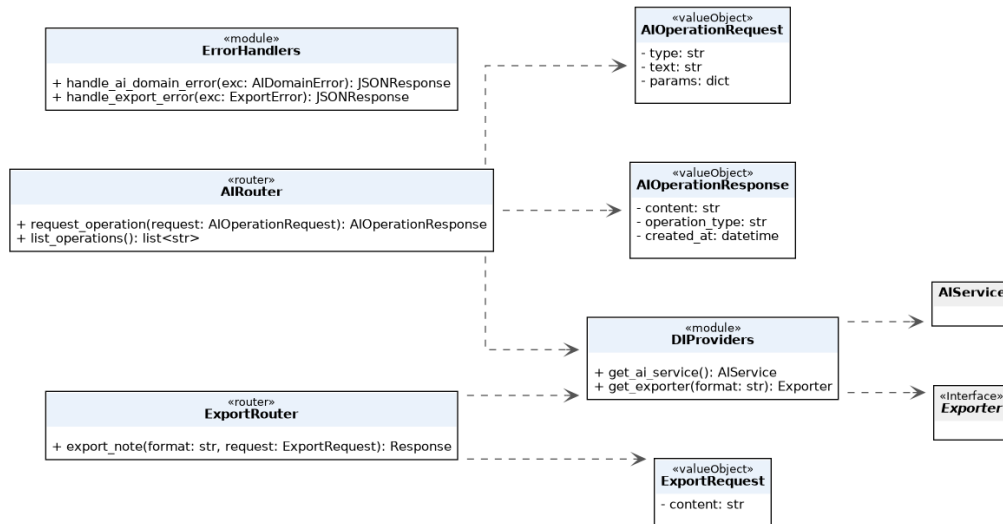


Figura 6: Diagramma delle classi del livello di presentazione e DI del Backend.

3.4.2 Livello di Business Logic (Core LLM)

L'**AIService** è il motore centrale del dominio AI: riceve il tipo di operazione richiesta (riassunto, traduzione, riscrittura, Distant Writing, o una delle sei prospettive dei “Cappelli di De Bono”), la risolve tramite un'**AIOperationFactory** auto-registrante, e ne invoca **build_prompt()** in modo polimorfico. Ogni operazione costruisce il proprio **Prompt** tramite un **PromptBuilder** fluente, isolando la logica specifica di ciascuna trasformazione testuale (Sezione 5, pattern Strategy, Factory Method e Builder).

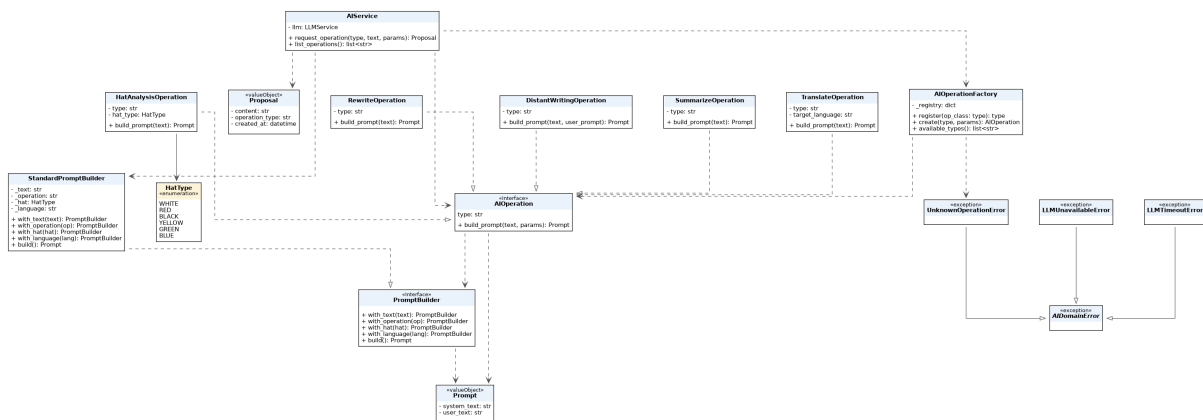


Figura 7: Diagramma delle classi: operazioni AI e costruzione dei prompt (AI.Domain/domain).

3.4.3 Livello di Integrazione / Infrastructure

Il pacchetto **AI.Domain/llm** maschera le specificità del provider LLM concreto dietro l'interfaccia **LLMService**. Una catena di decorator (**CachingLLMAdapter**, **LoggingLLMAdapter**) aggiunge funzionalità trasversali di caching e logging senza modificare l'implementazione di base; l'ultimo anello della catena è un adapter verso l'API compatibile con lo standard OpenAI (modelli Gemma, ChatGPT-OSS o Qwen, R2-V-O, R3-V-O), configurabile tramite **base_url** per puntare all'endpoint messo a disposizione dall'azienda proponente.

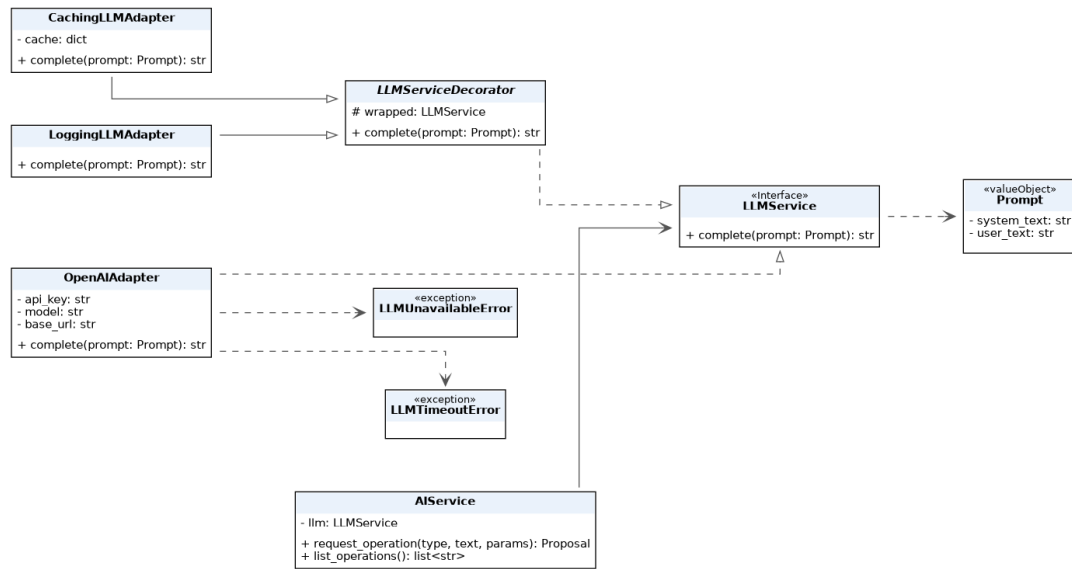


Figura 8: Diagramma delle classi: integrazione con il servizio LLM (AI_Domain/llm).

4 Interfaccia API e Comunicazione

Al fine di garantire il disaccoppiamento tra l'editor Markdown (Frontend) e l'elaboratore di prompt (Backend), la comunicazione avviene tramite API **REST** su JSON, esposta dal Backend FastAPI e consumata dal Frontend tramite il pattern Proxy (Sezione 5).

4.1 Struttura dei Payload AI

La comunicazione relativa alle operazioni di Intelligenza Artificiale (riassunto, traduzione, riscrittura, Sei Cappelli, Distant Writing) è normalizzata su un'unica struttura di richiesta e di risposta, validata da FastAPI tramite modelli tipizzati (`api/schemas`), indipendentemente dall'operazione richiesta.

DTO	Campo	Descrizione
AIOperationRequest	type	Tipo di operazione richiesta (es. <code>summarize</code> , <code>translate</code> , <code>rewrite</code> , <code>distant_writing</code> , <code>hat_white</code> , ...), risolto dinamicamente da <code>AIOperationFactory</code> .
	text	Testo selezionato dall'utente su cui operare (contesto).
	params	Parametri aggiuntivi specifici dell'operazione (es. lingua di destinazione per la traduzione, prompt utente per il Distant Writing).
AIOperationResponse	content	Testo generato dall'LLM, restituito come proposta.
	operation_type	Tipo di operazione a cui la risposta si riferisce, per consentire al Frontend di associare correttamente la proposta alla richiesta.
	created_at	Timestamp di generazione, utile per l'ordinamento e l'eventuale invalidazione della proposta.
ExportRequest	content	Contenuto Markdown della nota da convertire nel formato di esportazione richiesto (PDF, HTML o JSON).

Questa struttura unificata consente di aggiungere nuove operazioni AI (rispettando l'Open/Closed Principle, Sezione 5) senza introdurre nuovi endpoint o nuovi contratti di comunicazione: è sufficiente registrare una nuova strategia lato Backend.

4.2 Gestione degli Errori e Fallback

Le anomalie sono modellate come una gerarchia di eccezioni di dominio (`AIDomainError`), distinta dalle eccezioni tecniche di rete o di parsing:

- **LLMUnavailableError**: il servizio LLM esterno non è raggiungibile (assenza di connettività o servizio non configurato);
- **LLMTimeoutError**: la richiesta al modello LLM eccede il timeout configurato;
- **UnknownOperationError**: il tipo di operazione richiesto non è registrato in **AIOperationFactory**.

Un gestore di eccezioni centralizzato (**ErrorHandlers**) intercetta queste eccezioni all'interno dei router e le traduce in risposte HTTP normalizzate (codice di stato e messaggio), che il Frontend interpreta per notificare l'utente in caso di errore durante una qualsiasi delle operazioni AI (R49-F-O, R51-F-O, R54-F-O, R56-F-O, R58-F-O, R60-F-O, R62-F-O, R64-F-O, R66-F-O, R68-F-O), mantenendo il documento invariato e permettendo il rifiuto o la rigenerazione della proposta (R70-F-O, R71-F-O).

Coerentemente con il vincolo R6-V-O, ogni richiesta di elaborazione AI è indirizzata a un singolo modello LLM per l'intera durata della richiesta: il Backend non concatena chiamate a modelli differenti all'interno della stessa operazione.

5 Design Pattern Utilizzati

Di seguito sono illustrati i design pattern adottati per risolvere problematiche ricorrenti e garantire la scalabilità del codice di *Second Brain*. Ogni pattern è stato introdotto solo laddove i diagrammi architetturali ne mostrano un utilizzo concreto.

5.1 Pattern Creazionali

5.1.1 Factory Method (con auto-registrazione)

Problema risolto	AIService deve creare l'istanza dell'operazione AI corretta a partire da una stringa (il tipo di operazione richiesto dal Frontend) senza contenere un'istruzione condizionale per ogni tipo esistente, e nuove operazioni devono potersi aggiungere rispettando l'Open/Closed Principle.
Soluzione adottata	AIOperationFactory mantiene un registro interno e offre <code>register(op_class)</code> per l'auto-registrazione delle classi operazione e <code>create(type, params)</code> per istanziarle; solleva <code>UnknownOperationError</code> se il tipo non è registrato.
Utilizzo nel progetto	<code>AIService.request_operation</code> delega alla factory il compito di istanziare l'operazione corretta a partire dal campo <code>type</code> ricevuto dall'API, per ciascuna delle undici operazioni AI previste (riassunto, traduzione, riscrittura, Distant Writing, sei Cappelli).

5.1.2 Builder

Problema risolto	La costruzione di un <code>Prompt</code> richiede di comporre più parti opzionali (testo, tipo di operazione, cappello selezionato per l'analisi critica, lingua di destinazione per la traduzione) in modo incrementale, evitando costruttori con molti parametri opzionali.
Soluzione adottata	L'interfaccia <code>PromptBuilder</code> espone metodi fluenti (<code>with_text</code> , <code>with_operation</code> , <code>with_hat</code> , <code>with_language</code>) che restituiscono il builder stesso, e un metodo <code>build()</code> che produce il <code>Prompt</code> finale.
Utilizzo nel progetto	Ogni operazione AI usa il builder per assemblare il prompt in base ai soli parametri rilevanti: l'operazione di riassunto usa <code>with_text/with_operation/build</code> , mentre l'analisi secondo i Sei Cappelli aggiunge <code>with_hat</code> .

5.2 Pattern Strutturali

5.2.1 Decorator

Problema risolto	Al servizio di completamento LLM vanno aggiunte, in modo componibile e senza modificarne l'implementazione, funzionalità trasversali come il logging delle richieste/risposte e il caching dei risultati.
Soluzione adottata	<code>LLMServiceDecorator</code> è una classe astratta che realizza <code>LLMService</code> e mantiene un riferimento a un'istanza <code>wrapped: LLMService</code> ; i decorator concreti (<code>CachingLLMAdapter</code> , <code>LoggingLLMAdapter</code>) ne arricchiscono il comportamento prima/dopo aver delegato la chiamata.
Utilizzo nel progetto	La catena effettiva è <code>CachingLLMAdapter</code> → <code>LoggingLLMAdapter</code> → adapter verso il provider LLM, componibile e configurabile all'avvio dell'applicazione senza toccare <code>AIService</code> .

5.2.2 Adapter

Problema risolto	<code>AIService</code> deve rimanere indipendente dal provider LLM concreto; l'interfaccia esposta dal provider esterno (API compatibile con lo standard OpenAI) non coincide con l'interfaccia <code>LLMService</code> usata internamente.
Soluzione adottata	Un adapter realizza <code>LLMService</code> e incapsula i dettagli di connessione al provider (<code>api_key</code> , <code>model</code> , <code>base_url</code>), traducendo <code>complete(prompt)</code> in una richiesta HTTP verso il servizio LLM e gestendo le eccezioni specifiche (<code>LLMUnavailableError</code> , <code>LLMTimeoutError</code>).
Utilizzo nel progetto	Grazie a <code>base_url</code> configurabile, l'adapter permette di puntare indifferentemente ai modelli Gemma, ChatGPT-OSS o Qwen messi a disposizione dall'azienda proponente (R2-V-O, R3-V-O), senza modifiche al resto del sistema.

5.2.3 Proxy (remoto)

Problema risolto	Il Frontend deve invocare le funzionalità di note-taking e AI esposte dal Backend come se fossero locali, senza spargere nel codice applicativo i dettagli di trasporto (HTTP, base URL, serializzazione).
Soluzione adottata	Le interfacce <code>NoteService</code> e <code>AIService</code> (lato Frontend) definiscono le operazioni disponibili; le rispettive implementazioni Proxy le realizzano incapsulando la comunicazione con, rispettivamente, il file system locale e il Backend REST.
Utilizzo nel progetto	<code>NoteModel</code> dipende solo dall'interfaccia <code>NoteService</code> , <code>AIRequestModel</code> solo dall'interfaccia <code>AIService</code> : nessuna delle due classi di Model conosce se l'implementazione sia locale o remota, facilitando anche il testing tramite implementazioni fittizie.

5.3 Pattern Comportamentali

5.3.1 Strategy

Problema risolto	Le undici operazioni AI condividono la stessa interfaccia di utilizzo (costruzione di un <code>Prompt</code> a partire da un testo) ma differiscono nell'algoritmo di costruzione del prompt stesso; nuove operazioni devono potersi aggiungere senza modificare <code>AIService</code> .
Soluzione adottata	Un'interfaccia comune per le operazioni AI definisce <code>build_prompt(text, params)</code> ; ciascuna operazione concreta (riassunto, traduzione, riscrittura, Distant Writing, le sei varianti dell'analisi critica) la implementa con la propria logica.
Utilizzo nel progetto	<code>AIService</code> non conosce le operazioni concrete: riceve un'istanza dalla factory e la invoca in modo polimorfo, come illustrato nel diagramma di sequenza di Figura 10.

5.3.2 Command

Problema risolto	Le modifiche al testo della nota (formattazione, tabelle, elenchi, link, inserimento) devono poter essere annullate e ripetute in modo uniforme (R43-F-O, R44-F-O), senza che <code>NoteModel</code> conosca i dettagli di ciascuna operazione di editing.
Soluzione adottata	Un'interfaccia <code>EditCommand</code> definisce <code>execute()/undo()</code> ; ogni tipo di modifica la implementa incapsulando lo stato necessario per l'annullamento. Una <code>CommandHistory</code> mantiene due pile (undo/redo).
Utilizzo nel progetto	<code>NoteModel.executeCommand(c)</code> delega a <code>CommandHistory</code> , che esegue e archivia il comando; l'accettazione di una proposta AI riusa lo stesso meccanismo, inserendo il testo generato tramite un comando di inserimento.

5.3.3 Observer

Problema risolto	Model e View devono notificare i propri cambiamenti a più osservatori (View, Controller) senza conoscerne il tipo concreto, per mantenere il disaccoppiamento richiesto dal pattern MVC (Sezione 3.3).
Soluzione adottata	Un'interfaccia <code>Subject</code> definisce <code>attach/detach/notify</code> ; un'interfaccia <code>Observer</code> definisce <code>update()</code> . Il Model realizza <code>Subject</code> ; la View realizza sia <code>Subject</code> sia <code>Observer</code> ; il Controller realizza <code>Observer</code> .
Utilizzo nel progetto	Ogni modifica al contenuto della nota o allo stato di una richiesta AI propaga automaticamente l'aggiornamento a View e Controller, garantendo la visualizzazione continua dello stato richiesta da R72-F-O.

5.3.4 Dependency Injection

Problema risolto	I router FastAPI devono ottenere le proprie dipendenze (<code>AIService</code> , esportatori per formato) senza costruirle direttamente, per restare testabili e disaccoppiati dalla configurazione concreta (provider LLM, formato di esportazione).
Soluzione adottata	<code>DIProviders</code> espone funzioni fabbrica (<code>get_ai_service</code> , <code>get_exporter</code>) iniettate nei router dal meccanismo di Dependency Injection nativo di FastAPI (<code>Depends</code>).
Utilizzo nel progetto	<code>AIRouter</code> ed <code>ExportRouter</code> dipendono da <code>DIProviders</code> per ottenere le proprie collaborazioni a runtime, invece di istanziarle staticamente.

6 Flusso dei Dati (Data Flow)

Questa sezione illustra il ciclo di vita delle informazioni all'interno del sistema, partendo dall'interazione dell'utente fino all'ottenimento del risultato generato dall'Intelligenza Artificiale o alla persistenza della nota.

6.1 Flusso di esecuzione: Elaborazione AI (es. riassunto)

Il flusso è illustrato in due diagrammi di sequenza complementari: il tratto Frontend (Figura 9), dalla selezione del testo fino all'invio asincrono della richiesta, e il tratto REST → LLM (Figura 10), dal router REST fino al servizio LLM esterno, che attraversa in un unico flusso i pattern Strategy, Builder, Factory Method, Decorator e Adapter descritti in Sezione 5.

1. **Interazione Utente (Frontend):** l'utente seleziona una porzione di testo nell'editor Markdown e richiede un'operazione specifica (es. "Riassumi").
2. **Creazione Richiesta:** la View notifica il Controller, che invoca il metodo `requestAIOperation()` di `AIRequestModel`; il Model invia la richiesta al Backend tramite l'`AIServiceProxy` (chiamata asincrona), mentre l'interfaccia mostra uno stato di caricamento (R72-F-O).
3. **Validazione e Costruzione Prompt (Backend):** `AIRouter` riceve la richiesta e la inoltra ad `AIService`, che risolve la strategia corretta tramite `AIOperationFactory` e costruisce un `Prompt` strutturato tramite `PromptBuilder`.
4. **Chiamata Esterna:** `AIService` invoca `LLMService.complete()`; la richiesta attraversa la catena di decorator (cache, logging) fino all'adapter che contatta il servizio LLM esterno tramite l'API compatibile con lo standard OpenAI.
5. **Parsing e Restituzione:** il Backend costruisce una `Proposal` a partire dal testo ricevuto e la restituisce come `AIOperationResponse` al Frontend.
6. **Aggiornamento UI:** l'interfaccia mostra la proposta all'utente in un modale dedicato (R74-F-O), permettendone l'accettazione, il rifiuto o la rigenerazione (R69–R71).

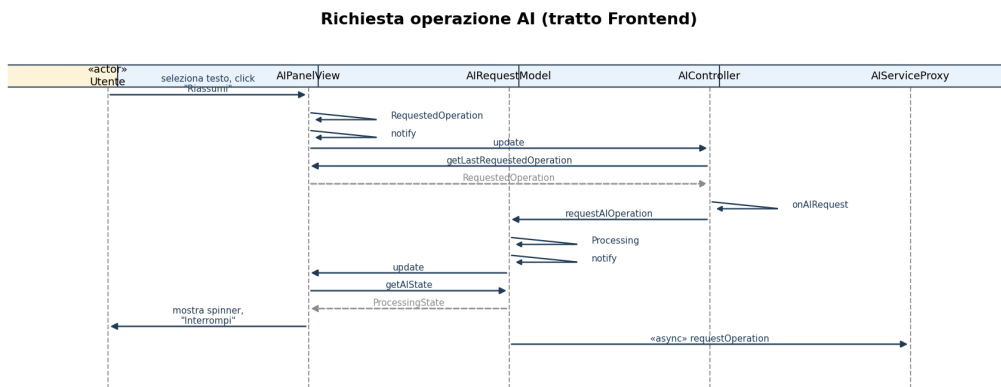


Figura 9: Diagramma di sequenza: richiesta di un'operazione AI, tratto Frontend.

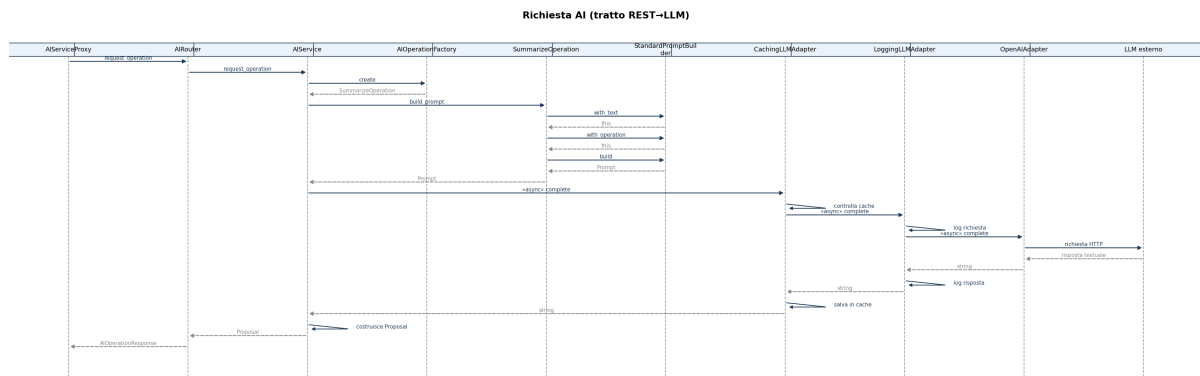


Figura 10: Diagramma di sequenza: richiesta di un'operazione AI, tratto REST → LLM.

6.2 Flusso di esecuzione: Editing con Undo

Il pattern Command consente di trattare in modo uniforme operazioni di editing eterogenee e il relativo annullamento. La Figura 11 mostra la formattazione di un testo selezionato e il successivo Undo: la View registra la richiesta dell'utente, si auto-notifica, il Controller la preleva e crea il comando corrispondente, che viene eseguito e archiviato da **CommandHistory**. L'Undo riesegue in ordine inverso l'ultimo comando in pila, ripristinando il contenuto precedente tramite lo stesso oggetto comando.

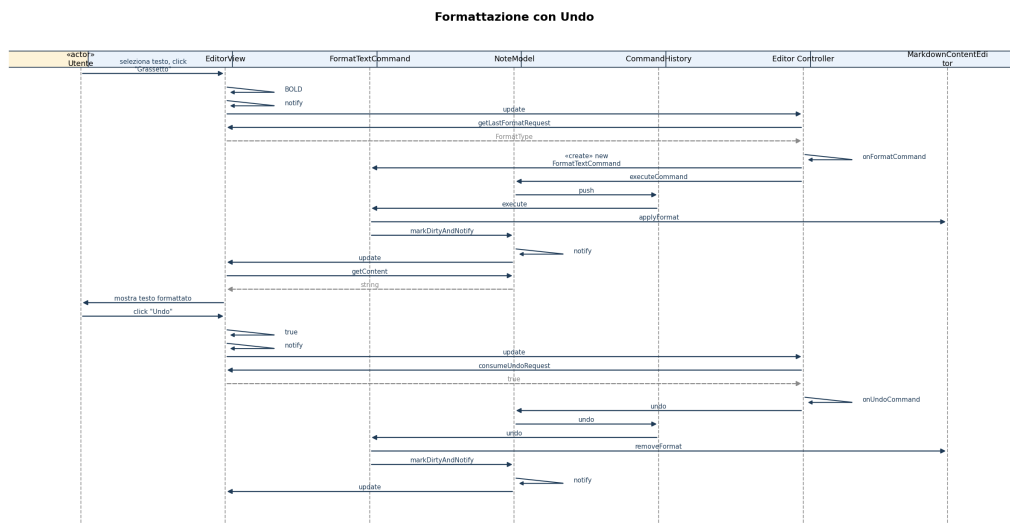


Figura 11: Diagramma di sequenza: formattazione di un testo e relativo Undo.

6.3 Flusso di esecuzione: Salvataggio locale della nota

La Figura 12 mostra il salvataggio della nota corrente (R75-F-O): il Controller richiama `NoteModel1.save()`, che delega tramite l'interfaccia `NoteService` al Proxy locale, responsabile dell'accesso al file system del browser. In caso di errore di I/O, l'eccezione risale fino alla View, che notifica l'utente mantenendo la nota invariata (R75-F-O).

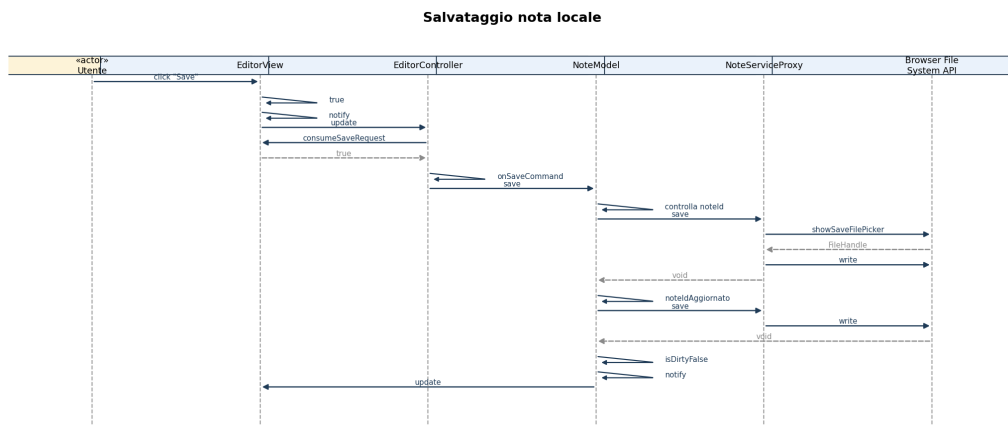


Figura 12: Diagramma di sequenza: salvataggio di una nota in locale.

7 Tracciamento e Stato dei Requisiti

Al fine di garantire che l'architettura progettata e le tecnologie scelte soddisfino le funzionalità concordate, si fornisce di seguito la tracciabilità tra le componenti architetturali descritte in questo documento e i requisiti individuati in [Analisi dei Requisiti](#) v1.8.0, Sezione 4 – Requisiti, a cui si rimanda per la descrizione testuale completa di ciascun requisito.

7.1 Tabella Tracciamento Componenti-Requisiti

Tabella 1: Tracciamento Componenti → Requisiti

Componente	Responsabilità	Requisiti coperti
MarkdownContent-Editor, EditorView/EditorController	Scrittura libera, copia/taglia/incolla, formattazione (grassetto, corsivo, sottolineato, barrato, citazione, intestazione), link, elenchi e tabelle	R1–R42
EditCommand (famiglia), CommandHistory	Annullamento e ripristino delle modifiche	R43-F-O, R44-F-O
EditorView (modalità di visualizzazione)	Visualizzazione esclusiva editor/render o affiancata (Split View)	R45–R47
AIRouter, AIService, AIOperationFactory, operazioni Strategy	Riassunto, traduzione, riscrittura, Distant Writing e analisi Sei Cappelli tramite LLM	R48, R50, R52, R53, R55, R57, R59, R61, R63, R65, R67
ErrorHandlers, gerarchia AIDomainError	Notifica degli errori di comunicazione con i servizi AI	R49, R51, R54, R56, R58, R60, R62, R64, R66, R68
AIRquestModel, AIPanelView, AIController	Accettazione/rifiuto/rigenerazione della proposta, interruzione manuale, stato di avanzamento continuo	R69–R74
NoteModel, NoteService/NoteServiceProxy	Salvataggio e caricamento della nota in formato Markdown dal file system locale	R75-F-O, R76-F-O
ExportRouter, esportatori per formato (Template Method)	Esportazione della nota in PDF, HTML e JSON	R77-F-O
Suite di test automatici, pipeline CI	Copertura di test, documentazione tecnica e di qualifica	R1-Q-O – R9-Q-D
Architettura Client-Server, adapter LLM	Esecuzione via browser, standard API OpenAI, modelli Gemma/ChatGPT-OSS/Qwen	R1-V-O – R7-V-O
Continua nella pagina successiva...		

Tabella 1 – Continua dalla pagina precedente

Componente	Responsabilità	Requisiti coperti
Rendering reattivo Vue.js, stato AIRequestState	Aggiornamento anteprima entro 1s, indicatore di caricamento continuo	R1-P-O, R2-P-O

I requisiti opzionali R4-V-Op e R78–R86 (persistenza remota, gestione utenti) non compaiono in questa tabella poiché il gruppo ha deciso di non implementarli in questa fase (si veda Sezione 7.2 per lo stato dettagliato e la motivazione): non esiste quindi alcun componente architetturale a cui tracciarli.

7.2 Stato di Soddisfacimento dei Requisiti

La tabella seguente riassume, per macro-gruppo, lo stato di implementazione dei requisiti funzionali, di qualità, di vincolo e prestazionali rispetto all'architettura descritta in questo documento. I gruppi ricalcano la struttura di [Analisi dei Requisiti](#), Sezione 4; il numero di requisiti obbligatori (90), desiderabili (2) e opzionali (12) è riepilogato nello stesso documento, Sezione 6 – Riepilogo.

I requisiti opzionali relativi alla persistenza remota e alla gestione utenti (R4-V-Op, R7-Q-Op, R8-Q-Op, R78–R86) sono stati valutati e volutamente esclusi dallo scope della release corrente: il gruppo ha scelto di concentrare le risorse disponibili sul consolidamento e sulla qualità dei requisiti obbligatori, rimandando un'eventuale implementazione a fasi successive del progetto qualora il tempo residuo lo consenta. Sono pertanto riportati con stato *Non implementato*.

Tabella 2: Stato dei Requisiti

Codici	Descrizione	Tipo	Stato
R1–R4	Scrittura, copia, taglia, incolla di testo nell'editor.	Obbligatorio	Soddisfatto
R5–R28	Attivazione / applicazione / disattivazione / rimozione delle formattazioni grassetto, corsivo, sottolineato, barrato, citazione e intestazione.	Obbligatorio	Soddisfatto
R29–R31	Inserimento, modifica e rimozione di link ipertestuali.	Obbligatorio	Soddisfatto
R32–R35	Inserimento, modifica, disattivazione e rimozione di elenchi puntati e numerati.	Obbligatorio	Soddisfatto
R36–R42	Inserimento e modifica di tabelle Markdown, con validazione dei parametri.	Obbligatorio	Soddisfatto
R43, R44	Annullamento (Undo) e ripristino (Redo) dell'ultima modifica.	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 2 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R45–R47	Visualizzazione esclusiva editor, render o affiancata (Split View).	Obbligatorio	Soddisfatto
R48, R49	Generazione riassunto tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R50, R51	Traduzione (Inglese, Francese, Spagnolo) tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R52	Traduzione in Tedesco come estensione facoltativa.	Desiderabile	Soddisfatto
R53, R54	Riscrittura del testo tramite LLM e notifica di errore.	Obbligatorio	Soddisfatto
R55, R56	Distant Writing da prompt utente e notifica di errore.	Obbligatorio	Soddisfatto
R57–R68	Analisi critica secondo le sei prospettive dei Cappelli di De Bono e relative notifiche di errore.	Obbligatorio	Soddisfatto
R69–R71	Accettazione, rifiuto e rigenerazione della proposta AI.	Obbligatorio	Soddisfatto
R72, R73	Visualizzazione continua dello stato di avanzamento e interruzione manuale della richiesta AI.	Obbligatorio	Soddisfatto
R74	Visualizzazione della proposta AI in un modale dedicato.	Obbligatorio	Soddisfatto
R75, R76	Salvataggio e caricamento della nota in formato Markdown su file system locale.	Obbligatorio	Soddisfatto
R77	Esportazione della nota in PDF, HTML e JSON.	Obbligatorio	In corso
R78–R86	Creazione/eliminazione nota, registrazione/autenticazione utente, gestione note remote.	Opzionale	Non implementato
R1-Q-O – R6-Q-O	Copertura di test, documentazione tecnica, versionamento Git, test di integrazione con i servizi LLM.	Obbligatorio	In corso
R7-Q-Op, R8-Q-Op	Test di integrazione e autenticazione del modulo opzionale di persistenza remota.	Opzionale	Non implementato
R9-Q-D	Adozione di una licenza open source permissiva.	Desiderabile	Soddisfatto
R1-V-O – R3-V-O	Esecuzione via browser standard e interazione con i modelli LLM tramite API compatibile OpenAI.	Obbligatorio	Soddisfatto
Continua nella pagina successiva...			

Tabella 2 – Continua dalla pagina precedente

Codici	Descrizione	Tipo	Stato
R4-V-Op	Persistenza remota tramite un database server-side.	Opzionale	Non implementato
R5-V-O – R7-V-O	Compatibilità browser, singolo modello LLM per richiesta, repository pubblico.	Obbligatorio	Soddisfatto
R1-P-O	Aggiornamento dell’anteprima Mark-down entro 1 secondo dalla modifica.	Obbligatorio	Soddisfatto
R2-P-O	Visualizzazione continua e reattiva dello stato di caricamento durante l’elaborazione AI.	Obbligatorio	Soddisfatto